

CO3-ASSESSMENT TOOL 1 – EXPERIMENTS

NAME:-T.NIKHITHA

REG NO:-192424114

Ex.1. Implement Merge Sort and analyze memory usage during execution. Record the auxiliary space

required for different input sizes. Interpret how additional memory impacts performance and justify the

trade-off between time and space complexity.

CODE:-

```
import sys

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
```

```
j += 1
```

```
while i < len(left):
```

```
    result.append(left[i])
```

```
    i += 1
```

```
while j < len(right):
```

```
    result.append(right[j])
```

```
    j += 1
```

```
return result
```

```
n = int(input("Enter number of elements: "))
```

```
arr = list(map(int, input("Enter elements: ").split()))
```

```
sorted_arr = merge_sort(arr)
```

```
print("Original array:", arr)
```

```
print("Sorted array:", sorted_arr)
```

```
print("Memory used by original array:", sys.getsizeof(arr), "bytes")
```

```
print("Memory used by sorted array:", sys.getsizeof(sorted_arr), "bytes")
```

```
print("Auxiliary space used:", sys.getsizeof(sorted_arr), "bytes")
```

INPUT:-

Enter number of elements: 6

Enter elements: 38 27 43 3 9 82

OUTPUT:-

Original array: [38, 27, 43, 3, 9, 82]

Sorted array: [3, 9, 27, 38, 43, 82]

Memory used by original array: 104 bytes

Memory used by sorted array: 120 bytes

Auxiliary space used: 120 bytes

Ex. 2 Design Binary Search and experimentally verify its logarithmic behavior by counting the number of iterations required to find elements in datasets of increasing size. Interpret the pattern observed and justify the efficiency compared to linear approaches.

CODE:-

```
def binary_search(arr, key):  
    low = 0  
    high = len(arr) - 1  
    count = 0  
  
    while low <= high:  
        count += 1  
        mid = (low + high) // 2  
  
        if arr[mid] == key:  
            return mid, count  
        elif arr[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    return -1, count  
  
n = int(input("Enter number of elements: "))  
arr = list(map(int, input("Enter sorted elements: ").split()))  
key = int(input("Enter element to search: "))  
  
pos, steps = binary_search(arr, key)
```

```
print("Array:", arr)
```

```
print("Element to search:", key)
```

```
if pos != -1:
```

```
    print("Element found at index", pos)
```

```
else:
```

```
    print("Element not found")
```

```
print("Number of iterations:", steps)
```

INPUT:-

Enter number of elements: 8

Enter sorted elements: 2 5 8 12 16 23 38 56

Enter element to search: 23

OUTPUT:-

Array: [2, 5, 8, 12, 16, 23, 38, 56]

Element to search: 23

Element found at index 5

Number of iterations: 3